```
# (1) tighten script 39: exact lattice index, exact composition identity, max
discrepancies with rtol=0 over ALL alignments and rephasings
# (2) count sign characters on the 14-rank relation lattice invariant under the
automorphisms of the pairwise table of the 24 weights.
import numpy as np, io, contextlib, itertools, sympy as sp, networkx as nx
from networkx.algorithms.isomorphism import GraphMatcher
with contextlib.redirect_stdout(io.StringIO()):
exec(open('39_robustness_of_T_sign_structure.py').read())
# ---- exact composition identity for both Cayley-Dickson conventions (integer
tables): sum_k (T[i,j,k]T[i',j',k]+T[i',j,k]T[i,j',k]) = 2 d_ii' d_jj'
for v in (0,1):
    Tv=tform(v); Ti=np.round(Tv).astype(int); assert np.abs(Tv-Ti).max()==0
    lhs=np.einsum('ijk,pqk->ijpq',Ti,Ti)+np.einsum('pjk,iqk->ijpq',Ti,Ti);
rhs=2*np.einsum('ip,jq->ijpq',np.eye(8,dtype=int),np.eye(8,dtype=int))
    print(f"convention {v}: composition identity holds exactly for all 8^4 index
combinations:",bool((lhs==rhs).all()))
# ---- exact lattice index
Cs=sp.Matrix(C0.tolist()); Co=sp.Matrix(Cold.tolist()); Q=(Co*Cs.T)*((Cs*Cs.T).inv())
print("exact: C_old = Q*C_sat:",bool(Q*Cs==Co)," Q integral:",all(x.q==1 for x in
Q)," det Q =",Q.det())
# ---- tolerance-enforced comparison over all alignments
def compare2(W1,Tw1,idx1,W2,Tw2,idx2):
    pos2={e:r for r,e in enumerate(idx2)}; n=0;bad=0;md=0;mr=0
    for tau,O in aligns(W1,W2):
        pi=[];ok=True
        for e in idx1:
            new=[None]*3
            for m in range(3): new[tau[m]]=widx(W2,tau[m],O@W1[m][e[m]])
            new=tuple(new)
            if None in new or new not in pos2: ok=False;break
            pi.append(new)
        n+=1
        if not ok or len(set(pi))!=len(idx1): bad+=1; continue
        a1=np.array([Tw1[e] for e in idx1]); a2=np.array([Tw2[p] for p in pi])
        f1=np.array([np.prod([a1[r]**int(c[r]) for r in range(len(idx1)) if c[r]!=0])
for c in C0]); f2=np.array([np.prod([a2[r]**int(c[r]) for r in range(len(idx1)) if
c[r]!=0]) for c in C0])
        md=max(md,np.abs(f1-f2).max()); mr=max(mr,rephase_residual(a1,a2,M0)[0])
    return n,bad,md,mr
print("\nall alignments, enforced tolerance (no relative slack):")
for name,Tc,seed in cases:
    d2,rk2,W2,Tw2,g2=pipeline(Tc,seed);
n,bad,md,mr=compare2(W0,Tw0,idx0,W2,Tw2,support(Tw2))
    print(f"  {name}: alignments {n}, support mismatches {bad}, max |invariant
difference| {md:.1e}, max rephasing residual {mr:.1e}")
# ---- table automorphisms of the 24 weights
```

```
Wall=np.vstack(W0); G=np.round(Wall@Wall.T,4); sets=[0]*8+[1]*8+[2]*8
print("\nrelation values between the 24 weights:",sorted(set(G.ravel())))
g=nx.complete_graph(24)
for i in range(24):
    for j in range(24):
        if i<j: g[i][j]["rel"]=float(G[i,j])
gm=GraphMatcher(g,g,edge_match=lambda a,b:abs(a["rel"]-b["rel"])<1e-6)
auts=[tuple(m[i] for i in range(24)) for m in gm.isomorphisms_iter()]
print("automorphisms of the unlabelled 24x24 relation table:",len(auts))
def info(p):
    img=[set(sets[p[i]] for i in range(8*m,8*m+8)) for m in range(3)]
    if all(len(x)==1 for x in img): return tuple(next(iter(x)) for x in img)
    return None
valid=[(p,info(p)) for p in auts]; print("  of these, preserving the partition into
the three weight sets:",sum(1 for _,t in valid if t is not None))
pos={e:r for r,e in enumerate(idx0)}
def Bmat(p,tau):
    pi={}
    for e in idx0:
        new=[None]*3
        for m in range(3): new[tau[m]]=p[8*m+e[m]]-8*tau[m]
        new=tuple(new);
        if new not in pos: return None
        pi[e]=new
    cols=[]
    for c in C0:
        pc=np.zeros(len(idx0),int)
        for e in idx0: pc[pos[pi[e]]]=c[pos[e]]
        x=np.rint(np.linalg.lstsq(C0.T.astype(float),pc.astype(float),rcond=None)
[0]).astype(int)
        if not (x@C0==pc).all(): return None
        cols.append(x)
    return np.array(cols).T    # column j = coordinates of image of basis vector j
xT=((np.round(f0.real)).astype(int)==-1).astype(int)
def f2null(rows):
    A=np.array(rows)%2; r=0; A=A.copy()
    for c in range(14):
        piv=[i for i in range(r,len(A)) if A[i,c]]
        if not piv: continue
        A[[r,piv[0]]]=A[[piv[0],r]]
        for i in range(len(A)):
            if i!=r and A[i,c]: A[i]^=A[r]
        r+=1
    return 14-r
def run(label,sel):
    rows=[];ok=0;fail=0;inv=True
    for p,t in sel:
        B=Bmat(p,t)
        if B is None: fail+=1; continue
        ok+=1; rows.extend(((B.T-np.eye(14,dtype=int))%2).tolist())
```

```
        if ((((B.T-np.eye(14,dtype=int))@xT)%2)!=0).any(): inv=False
    d=f2null(rows) if rows else 14
    print(f"  {label}: automorphisms used {ok} (failed to act {fail}); free dimension
d = {d} -> {2**d} invariant sign characters; chi_T invariant under all of them:
{inv}")
print("\ncounting invariant sign characters (L = rank 14, characters = F_2^14):")
V=[(p,t) for p,t in valid if t is not None]
run("all partition-preserving table automorphisms",V)
run("only those fixing each weight set (slot permutation trivial)",[(p,t) for p,t in
V if t==(0,1,2)])
run("only the cyclic slot permutations",[(p,t) for p,t in V if t in [(0,1,2),(1,2,0),
(2,0,1)]])
```